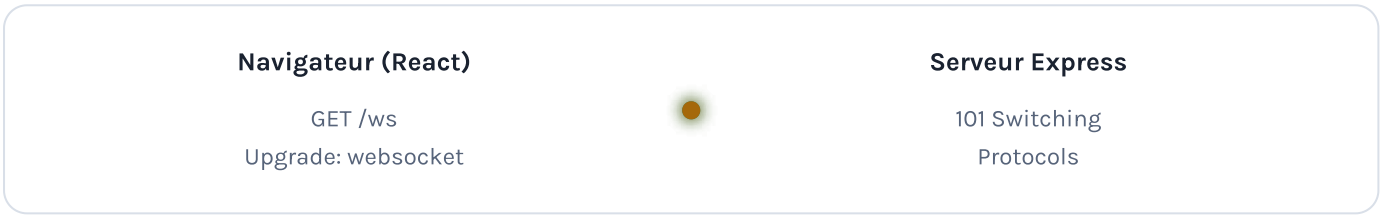
● CONNECTION: OPEN

WebSocket pour ton quiz

Fiche pas à pas pour comprendre et implémenter les WebSockets en Node.js, appliquée directement à ton backend `Express + https + ws` et à ton fichier `websocket/gameSocket.js` (actuellement vide, TODO : connexion/déconnexion, diffusion des coups en live, chat temps réel).



01 Pourquoi pas du HTTP classique

En HTTP, c'est toujours le client qui parle en premier : il envoie une requête, le serveur répond, la connexion se referme (ou reste en attente sans rien faire de plus). Pour un quiz en live, ça pose un problème très concret : quand le serveur décide qu'une nouvelle question commence, ou quand un autre joueur répond, **c'est le serveur qui doit prévenir tout le monde** — pas le client qui doit deviner qu'il faut redemander.

APPROCHE	SENS	LATENCE / COÛT	POUR TON QUIZ
Polling (setInterval + fetch)	Client → Serveur	Une requête HTTP toutes les X secondes, même sans rien de neuf	Le décalage entre joueurs sur le timer devient vite visible, et ça sature ton API pour rien
Server-Sent Events	Serveur → Client seul	Connexion HTTP tenue ouverte, faible latence	Marche pour diffuser les questions, mais un joueur ne peut pas répondre par le même canal
WebSocket	Bidirectionnel, full-duplex	Une seule connexion TCP, aucune re-négociation	Le serveur pousse la question à tous, chaque client pousse sa réponse — sur le même canal

À RETENIR

Une WebSocket, ce n'est pas « une requête HTTP qui dure longtemps ». C'est une requête HTTP qui sert une fois à négocier une connexion, puis qui se transforme en canal TCP brut, bidirectionnel, que les deux côtés peuvent utiliser librement jusqu'à ce que l'un des deux le ferme.

02 Le vocabulaire essentiel

Cinq mots à connaître avant d'écrire la moindre ligne de code.

TERME	SIGNIFICATION
Handshake	La requête HTTP initiale avec l'en-tête `Upgrade: websocket`. Le serveur répond `101 Switching Protocols` s'il accepte. Après ça, ce n'est plus du HTTP.
wss:// vs ws://	`wss` est à `ws` ce que `https` est à `http`. Ton serveur tourne déjà en HTTPS (certs dans `Backend/certs`), donc ta WebSocket sera automatiquement en `wss://`.
readyState	L'état d'une connexion côté client : `CONNECTING` (0), `OPEN` (1), `CLOSING` (2), `CLOSED` (3). Toujours vérifier `OPEN` avant d'envoyer.
Ping / Pong	Deux trames de contrôle invisibles pour ton code applicatif, utilisées pour vérifier que l'autre bout est toujours là (voir section 08).
Close code	`1000` = fermeture normale, `1006` = coupure anormale (réseau, crash). Les codes `4000`–`4999` sont libres, tu peux les utiliser pour tes propres erreurs applicatives.

03 Choisir sa librairie : ws vs socket.io

Ton `package.json` backend n'a ni `ws` ni `socket.io` pour l'instant. Voici pourquoi partir sur `ws` a du sens pour ton projet.

	WS	SOCKET.IO
Ce que c'est	Implémentation fine du protocole WebSocket standard, rien de plus	Framework au-dessus de WebSocket, avec son propre protocole de message
Client requis	L'API `WebSocket` native du navigateur suffit, zéro dépendance côté React	Nécessite le package `socket.io-client`
Rooms, reconnexion, heartbeat	À coder soi-même (ce que fait cette fiche)	Fournis clé en main
Ce que tu comprends	Tout, parce que tu écris tout	Ce que la lib décide de t'exposer
Intégration à ton `https.createServer`	Directe, même port, même certificat	Possible mais rajoute une couche

RECOMMANDATION

`ws` pour ft_transcendence : léger, tu gardes exactement ton `https.createServer(options, app)` actuel, et en soutenance tu peux expliquer chaque ligne — il n'y a pas de boîte noire entre toi et le protocole.

terminal — dans Backend/

```
npm install ws
```

04 Brancher ws sur ton serveur Express/HTTPS

Règle d'or : **un seul serveur HTTP**. Tu ne dois pas ouvrir un deuxième port pour les WebSockets — tu attaches `ws` au *même* serveur HTTPS qui sert déjà Express, en écoutant son événement `'upgrade'`.

Backend/server.js

```
// ... tes require() existants (express, cors, cookieParser, https, fs...) ...
const { WebSocketServer } = require('ws');
const { authenticateUpgrade } = require('./websocket/auth');
const { handleConnection } = require('./websocket/gameSocket');

// ... app.use(...) et tes routes existantes ne changent pas ...

// avant : https.createServer(options, app).listen(...)
// on garde la référence au serveur pour pouvoir écouter 'upgrade'
const server = https.createServer(options, app);

const wss = new WebSocketServer({ noServer: true });

server.on('upgrade', (req, socket, head) => {
  // on ne gère que les connexions WS sur /ws, le reste (HMR de Vite en
  // dev, etc.) ne nous concerne pas
  if (!req.url.startsWith('/ws')) {
    socket.destroy();
    return;
  }

  const user = authenticateUpgrade(req); // section 05
  if (!user) {
    socket.write('HTTP/1.1 401 Unauthorized\r\n\r\n');
    socket.destroy();
    return;
  }

  wss.handleUpgrade(req, socket, head, (ws) => {
    ws.user = user;
    wss.emit('connection', ws, req);
  });
});

wss.on('connection', handleConnection); // section 09

server.listen(process.env.PORT, () => {
  console.log(`Server running on https://localhost:${process.env.PORT}`);
});
```

POURQUOI NOSERVER: TRUE

Avec `noServer: true`, `ws` ne bind aucun port lui-même : c'est toi qui décides, dans `server.on('upgrade')`, si une connexion a le droit d'être acceptée. C'est ce qui te permet de vérifier le JWT *avant* de répondre `101` — un client non authentifié n'obtient jamais de WebSocket.

05 Authentifier la connexion avec ton JWT

L'API `WebSocket` du navigateur ne permet pas d'ajouter des en-têtes personnalisés (impossible d'envoyer un `Authorization: Bearer ...`). Bonne nouvelle : ton token est

déjà dans un cookie `httpOnly` nommé `token` (posé par `auth.controller.js` via `generateToken`), et **les cookies sont envoyés automatiquement** pendant le handshake, puisque c'est encore une requête HTTP classique à ce moment-là.

Backend/websocket/auth.js

```
const jwt = require('jsonwebtoken');

function parseCookies(header = '') {
  return Object.fromEntries(
    header.split(';')
      .filter(Boolean)
      .map((pair) => {
        const [key, ...rest] = pair.trim().split('=');
        return [key, decodeURIComponent(rest.join('='))];
      })
  );
}

function authenticateUpgrade(req) {
  const cookies = parseCookies(req.headers.cookie);
  const token = cookies.token;
  if (!token) return null;

  try {
    return jwt.verify(token, process.env.JWT_SECRET); // { id, email }
  } catch {
    return null;
  }
}

module.exports = { authenticateUpgrade };
```

PIÈGE SPÉCIFIQUE À TON SETUP

Ton cookie est posé avec `sameSite: 'lax'` (`lib/cookieOptions.js`). En dev, ton frontend Vite tourne sur un port différent de ton backend HTTPS (`vite.config.js` n'a pas encore de proxy) : ce sont donc deux origines différentes. Un cookie `SameSite=Lax` **n'est pas envoyé** sur une requête cross-site déclenchée par du JS (fetch, XHR, WebSocket) — seulement sur une navigation top-level. Résultat probable : `ws.user` sera `undefined` côté serveur en dev, même avec un token valide.

La solution la plus simple : proxifier Vite

Fais passer les requêtes `/api` et `/ws` du dev server Vite vers ton backend : le navigateur ne voit alors qu'une seule origine, et le cookie part normalement.

Frontend/vite.config.js

```
export default defineConfig({
  plugins: [react()],
  server: {
    proxy: {
      '/api': { target: 'https://localhost:3000', secure: false, changeOrigin: true },
      '/ws':  { target: 'https://localhost:3000', secure: false, changeOrigin: true, ws: true },
    },
  },
});
```

`secure: false` ici sert seulement à accepter ton certificat auto-signé de dev côté proxy Vite — pas à désactiver `wss`. En production, si le frontend est servi par le même serveur/reverse-proxy que l'API, ce problème disparaît de lui-même.

06 Concevoir un protocole de messages

`ws` transporte du texte brut : c'est à toi d'inventer une convention. La plus simple et la plus robuste : chaque message est un objet JSON avec un champ `type` qui dit ce qu'il contient.

forme générale

```
{ "type": "submit_answer", "payload": { "questionId": 12, "answerId": 3 } }
```

TYPE	SENS	PAYLOAD
join_room	`client → serveur`	`{ roomId }`
player_joined	`serveur → tous`	`{ userId, email }`
question	`serveur → tous`	`{ index, text, choices, timeLimitMs }`
submit_answer	`client → serveur`	`{ questionId, answerId }`
leaderboard_update	`serveur → tous`	`{ players: [{ id, email, score }] }`
chat_message	`↔`	`{ userId?, text, ts? }`
player_left	`serveur → tous`	`{ userId }`
error	`serveur → client seul`	`{ message }`

BONNE PRATIQUE

Toujours entourer le `JSON.parse` d'un `try/catch` côté serveur. Un client (ou un outil comme `wscat`) peut envoyer n'importe quoi — un texte mal formé ne doit jamais faire planter ton process.

07 Gérer les rooms (une partie = une room)

Une « room » regroupe les joueurs d'une même partie de quiz. En mémoire, une `Map` suffit largement pour un projet comme le tien (un seul process Node).

Backend/websocket/rooms.js

```
const rooms = new Map(); // roomId → { players, currentQuestionIndex, scores }

function getOrCreateRoom(roomId) {
  if (!rooms.has(roomId)) {
    rooms.set(roomId, {
      players: new Map(),        // userId → ws
      currentQuestionIndex: -1,
      scores: new Map(),         // userId → score
    });
  }
  return rooms.get(roomId);
}

function broadcast(roomId, message, exceptUserId = null) {
  const room = rooms.get(roomId);
  if (!room) return;

  const data = JSON.stringify(message);
  for (const [userId, client] of room.players) {
    if (userId === exceptUserId) continue;
    if (client.readyState === client.OPEN) client.send(data);
  }
}

module.exports = { rooms, getOrCreateRoom, broadcast };
```

LIMITE À CONNAÎTRE

Cet état vit dans la RAM d'un seul process. Si un jour tu fais tourner plusieurs instances du backend derrière un load balancer, deux joueurs de la même room peuvent finir sur deux process différents qui ne se parlent pas. Pour ft_transcendence en un seul container, ce n'est pas un problème — à garder en tête si le projet grossit (voir section 13).

08 Heartbeat : détecter les déconnexions

Un laptop qu'on ferme brutalement ou un wifi qui tombe ne déclenche pas toujours l'événement `'close'` côté serveur — TCP peut mettre plusieurs minutes à s'en rendre

compte. Sans heartbeat, un joueur « fantôme » reste dans ta room, fausse ton classement et ne libère jamais sa place.

Backend/websocket/gameSocket.js (extrait)

```
function heartbeat() { this.isAlive = true; }

wss.on('connection', (ws) => {
  ws.isAlive = true;
  ws.on('pong', heartbeat);
  // ... handleConnection(ws, req) ...
});

const interval = setInterval(() => {
  wss.clients.forEach((ws) => {
    if (!ws.isAlive) return ws.terminate(); // déclenche 'close' → cleanup room
    ws.isAlive = false;
    ws.ping();
  });
}, 30000);

wss.on('close', () => clearInterval(interval));
```

Toutes les 30 secondes, le serveur envoie un `ping` à chaque client encore marqué vivant. Le navigateur répond automatiquement par un `pong` (rien à coder côté client) : ça remet `isAlive` à `true`. Si un client n'a pas répondu depuis le tick précédent, on coupe la connexion nous-mêmes.

09 Implémenter gameSocket.js pour de vrai

En assemblant les sections 05 à 08, voici de quoi remplacer le TODO actuel de `websocket/gameSocket.js` par une implémentation qui couvre exactement ce qu'il annonçait : connexion/déconnexion, diffusion en live, chat temps réel.

Backend/websocket/gameSocket.js

```
const { getOrCreateRoom, broadcast, rooms } = require('./rooms');

function handleConnection(ws, req) {
  console.log(`Nouvelle connexion WS : user #${ws.user.id}`);

  ws.on('message', (raw) => {
    let message;
    try {
      message = JSON.parse(raw);
    } catch {
      return ws.send(JSON.stringify({ type: 'error', payload: { message: 'JSON invalide' } }));
    }

    switch (message.type) {
      case 'join_room':      return onJoinRoom(ws, message.payload);
      case 'submit_answer':  return onSubmitAnswer(ws, message.payload);
      case 'chat_message':   return onChatMessage(ws, message.payload);
      default:
        ws.send(JSON.stringify({ type: 'error', payload: { message: `Type inconnu: ${message.type}` }
}));
    }
  });

  ws.on('close', () => onDisconnect(ws));
  ws.on('error', (err) => console.error('WS error:', err.message));
}

function onJoinRoom(ws, { roomId }) {
  const room = getOrCreateRoom(roomId);
  room.players.set(ws.user.id, ws);
  ws.roomId = roomId; // pratique pour le cleanup au close

  broadcast(roomId, {
    type: 'player_joined',
    payload: { userId: ws.user.id, email: ws.user.email },
  });
}

function onSubmitAnswer(ws, { questionId, answerId }) {
  // TODO: vérifier la bonne réponse via Prisma (modèle Answer.isCorrect),
  // mettre à jour room.scores, puis broadcast('leaderboard_update', ...)
}

function onChatMessage(ws, { text }) {
  if (!ws.roomId || typeof text !== 'string' || text.trim() === '') return;
  broadcast(ws.roomId, {
    type: 'chat_message',
    payload: { userId: ws.user.id, text: text.slice(0, 300), ts: Date.now() },
  });
}

function onDisconnect(ws) {
  if (!ws.roomId) return;
  const room = rooms.get(ws.roomId);
  if (!room) return;
```

```
  room.players.delete(ws.user.id);
  broadcast(ws.roomId, { type: 'player_left', payload: { userId: ws.user.id } });

  if (room.players.size === 0) rooms.delete(ws.roomId); // plus personne → on libère la mémoire
}

module.exports = { handleConnection };
```

POURQUOI ONSUBMITANSWER RESTE UN TODO

La plomberie (protocole, rooms, broadcast, cleanup) est prête. La logique de correction dépend de ton schéma Prisma (`Quiz` , `Question` , `Answer` , `QuizAttempt` , `UserAnswer`) et des règles de score que tu veux appliquer — c'est la partie vraiment spécifique à ton jeu, à écrire toi-même en t'appuyant sur ces modèles.

10 Le client côté React

Un hook simple : une connexion par montage du composant, reconnexion avec backoff si elle tombe, nettoyage au démontage. L'API `WebSocket` native envoie les cookies du même domaine automatiquement — pas besoin d'option `credentials` comme avec `fetch` .

Frontend/src/hooks/useGameSocket.js

```
import { useEffect, useRef, useState, useCallback } from 'react';

export function useGameSocket(roomId) {
  const wsRef = useRef(null);
  const [messages, setMessages] = useState([]);
  const [connected, setConnected] = useState(false);

  useEffect(() => {
    if (!roomId) return;
    let attempt = 0;
    let cancelled = false;
    let socket;

    const connect = () => {
      // même origine grâce au proxy Vite → le cookie 'token' part tout seul
      socket = new WebSocket(`wss://${window.location.host}/ws`);
      wsRef.current = socket;

      socket.onopen = () => {
        attempt = 0;
        setConnected(true);
        socket.send(JSON.stringify({ type: 'join_room', payload: { roomId } }));
      };

      socket.onmessage = (event) => {
        const message = JSON.parse(event.data);
        setMessages((prev) => [...prev, message]);
      };

      socket.onclose = () => {
        setConnected(false);
        if (cancelled) return;
        const delay = Math.min(1000 * 2 ** attempt, 10000); // backoff exponentiel, max 10s
        attempt += 1;
        setTimeout(connect, delay);
      };
    };

    connect();
    return () => {
      cancelled = true;
      socket?.close();
    };
  }, [roomId]);

  const send = useCallback((type, payload) => {
    const socket = wsRef.current;
    if (socket?.readyState === WebSocket.OPEN) {
      socket.send(JSON.stringify({ type, payload }));
    }
  }, []);

  return { connected, messages, send };
}
```

11 Tester sans le frontend

Avant de brancher React, valide ton serveur avec `wscat` en ligne de commande — bien plus rapide pour itérer sur le protocole.

terminal

```
npm install -g wscat

# le cookie n'est pas envoyé par wscat → passe le token autrement pour tester,
# ou teste temporairement une route qui accepte un token en query string
wscat -c wss://localhost:3000/ws -H "Cookie: token=TON_JWT_ICI" --no-check

# une fois connecté, envoie un message brut :
> {"type":"join_room","payload":{"roomId":"room-1"}}
```

`--no-check` ignore la validité du certificat auto-signé en local. Récupère un JWT valide en te connectant via `POST /api/auth/login` (ou équivalent) et en copiant la valeur du cookie `token` depuis les DevTools.

12 Pièges classiques à éviter

1. **Ne jamais faire confiance à un `userId` envoyé dans le payload.** Utilise toujours `ws.user.id`, fixé une fois pour toutes au handshake — jamais une valeur reçue dans un message.
2. **Toujours vérifier `readyState === OPEN` avant `send()`.** Une socket en train de se fermer peut encore traîner dans ta `Map` un court instant.
3. **Nettoyer la room au `'close'`, pas seulement au `'error'`.** Les deux événements existent, mais seul `'close'` est garanti à chaque fin de connexion.
4. **Limiter la taille et la fréquence des messages entrants** (ex. tronquer le chat, ignorer les `submit_answer` répétés pour la même question) pour éviter qu'un client buggé ou malveillant ne spamme tes autres joueurs.
5. **Ne pas bloquer l'event loop** dans un handler de message (pas de boucle lourde ni d'opération Prisma synchrone-bloquante) — ça gèlerait toutes les autres connexions WS le temps du calcul.
6. **Vérifier l'origine de la connexion** (`req.headers.origin`) en plus du JWT si tu veux te prémunir d'un cross-site WebSocket hijacking — le JWT suffit déjà largement ici, mais c'est bon à savoir.

13 Pour aller plus loin

- **Multi-instances :** si le backend tourne un jour sur plusieurs process/containers, l'état en `Map` ne suffit plus — un pub/sub partagé (Redis) permet de diffuser un message à

des clients connectés sur d'autres process.

- **Namespaces / rooms avancées :** si la gestion manuelle des rooms devient complexe (plusieurs types de parties, lobbies, matchmaking), c'est le moment où `socket.io` peut redevenir pertinent — mais seulement une fois que tu maîtrises ce que fait `ws` tout seul.
- **Compression des messages :** l'option `perMessageDeflate` de `ws`, utile si le chat ou les payloads grossissent.

14 Checklist avant de coder

- [ ] `npm install ws` fait dans `Backend/`
- [ ] `server.js` garde une référence `server` et écoute `'upgrade'`
- [ ] `websocket/auth.js` vérifie le JWT avant tout `handleUpgrade`
- [ ] Proxy Vite configuré pour que le cookie `token` parte bien en dev
- [ ] Protocole de messages (`type` + `payload`) écrit noir sur blanc, pas juste dans ta tête
- [ ] Heartbeat ping/pong en place, testé en coupant le wifi d'un client
- [ ] Cleanup de room testé : un joueur qui ferme l'onglet disparaît bien du classement

---

Fiche générée pour le backend `ft_transcendence` (Express + Prisma + ws) — pense à relire chaque extrait avant de le coller, certains noms de fichiers supposent une organisation que tu es libre d'adapter.